Разработка Java EE — ответы на вопросы 1–3

По конспекту лекций «Разработка Java-приложений» (автор — И. Лёзина). Темы: распределённые системы и JDBC.

Как читать этот документ. Каждый пункт состоит из двух частей: 📘 **По пособию** — техническая формулировка, близкая к тексту конспекта (её удобно говорить на экзамене), и 💡 **Простыми словами** — объяснение на понятных примерах. Схемы расположены после текста каждого вопроса.

Вопрос 1. Распределённая система: причины создания, принципы построения, последствия распределённости, требования

1.1. Что такое распределённая система

📘 **По пособию.** Распределённая система — это собрание независимых компьютеров (узлов), соединённых сетью и программным обеспечением, обеспечивающим их совместное функционирование. Узлы автономны, а взаимодействие между ними осуществляется посредством передачи сообщений.

💡 **Простыми словами.** Это несколько отдельных компьютеров, которые работают как одна система и решают общую задачу. Типичный пример — интернет-магазин: ваш браузер (клиент), веб-сервер, сервер приложений и сервер базы данных — это разные машины, но для покупателя всё выглядит как один сайт.

Три признака, по которым можно узнать распределённую систему:

- **узлы независимы** — у каждого своя память, свои процессор и часы;
- **связь только через сеть** — общих переменных нет, узлы обмениваются сообщениями;
- **общая цель** — специальное программное обеспечение (middleware) заставляет узлы работать совместно.

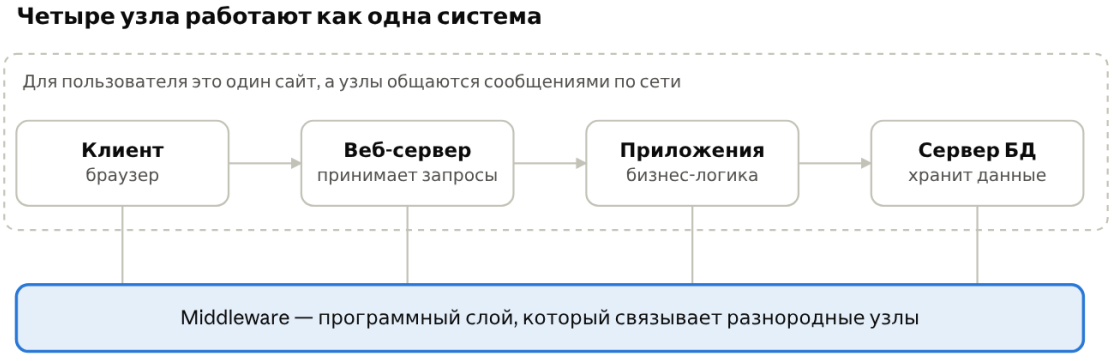


схема · 4 узла и middleware

Схема 1. Узлы — отдельные компьютеры, но пользователь видит единый сайт.

1.2. Причины создания распределённых систем

В пособии названы две группы причин.

Причина 1. Совместное использование общих ресурсов (данных, устройств, приложений).

💡 Один склад данных, один принтер, одно приложение — на всех сотрудников. Нет смысла копировать базу клиентов на каждый компьютер: проще хранить её на сервере, а остальные пусть обращаются по сети.

Причина 2. Улучшение функциональных характеристик системы.

Что улучшается	📘 Формулировка пособия	💡 Пример
Производительность	Нагрузка делится между узлами	Запросы обслуживают 5 серверов, а не один
Надёжность и устойчивость к сбоям	Отказ одного узла не останавливает систему	Упал один сервер — остальные продолжают работать
Упрощение и удешевление за счёт специализации	Каждый компонент выполняет свою часть работы	Отдельный сервер только для БД, отдельный только для картинок
Отношение цена/производительность	Несколько недорогих узлов дешевле одного мощного	Пять обычных серверов обходятся дешевле суперкомпьютера

1.3. Принципы построения

Пособие выделяет три принципа.

1) Функциональное разделение. Узлы выполняют *разные* задачи: клиент / сервер, хост / терминал, сбор данных / обработка данных. Решение — создание *разделяемых сервисов*, которые управляют набором ресурсов и предоставляют услуги пользователям.

💡 Как в ресторане: повар готовит, официант обслуживает, кассир принимает оплату. Каждый занят своим делом, а вместе они обслуживают гостя.

2) Естественное разделение. Разделение определяется самой задачей (система обслуживания сети супермаркетов, сеть для коллективной работы). Однотипные узлы несут *одинаковую* функциональную нагрузку.

💡 Сеть супермаркетов: в каждом магазине стоит одинаковая касса с одинаковой программой. Распределённость «получилась сама», потому что магазины находятся в разных местах.

3) Балансировка нагрузки. Функциональность может *повторяться* на нескольких узлах. Задачи назначаются на процессоры так, чтобы нагрузка была равномерной. Это обеспечивает наибольшую эффективность использования вычислительных ресурсов.

💡 Несколько одинаковых касс и администратор в зале, который направляет покупателей туда, где очередь короче.

Принцип	Узлы делают	Ключевая идея	Пример
Функциональное разделение	Разное	Каждый узел — специалист	Клиент и сервер БД
Естественное разделение	Одно и то же, но в разных местах	Структуру задаёт сама задача	Магазины сети
Балансировка нагрузки	Одно и то же	Равномерно загрузить всех	Кластер веб-серверов



схема · 3 принципа построения

Схема 2. Принципы построения: кто что делает и как распределяется нагрузка.

1.4. Последствия распределённости

Распределённость даёт преимущества, но и порождает проблемы. Пособие называет четыре последствия.

1) Параллельность. Узлы работают одновременно. Отсюда вопросы синхронизации и «гонки потоков», коллизии при совместном доступе к данным, сервисам и устройствам, а также типичные проблемы: взаимные блокировки (deadlock) и ненадёжные коммуникации.

💡 Два кассира одновременно продают последний билет: если не договориться, билет продадут дважды (гонка). А deadlock — это два человека в узком проходе, каждый ждёт, пока другой отойдёт. В Java эти же проблемы возникают при работе с потоками (`Thread`, `synchronized`).

2) Нет «глобального» времени. Сообщения передаются асинхронно, а точность синхронизации часов ограничена.

💡 Часы на разных серверах всегда чуть расходятся. Поэтому нельзя надёжно сказать, какое из двух событий на разных машинах произошло раньше.

3) Нет состояния системы. В распределённой системе нет ни одного процесса, который знал бы текущее *глобальное* состояние системы. Это следствие параллелизма и самого механизма передачи данных.

💡 Пока сообщение «у меня осталось 10 штук товара» идёт по сети, на складе могло остаться уже 8. Целостной «фотографии» всей системы в конкретный момент никто не имеет.

4) Сбои. Процессы выполняются автономно и изолированно, поэтому неудачи отдельных процессов могут остаться необнаруженными, а сами процессы могут не подозревать об общесистемном сбое. Сбои происходят чаще, чем в централизованной системе, появляются новые причины сбоев, а сетевые сбои разбивают систему на изолированные части.

💡 Вы отправили запрос и не получили ответа. Сервер упал? Сеть оборвалась? Или он просто очень занят? Отличить эти случаи из одной точки часто невозможно.

1.5. Требования к распределённым системам

Пособие перечисляет семь требований: **открытость, безопасность, масштабируемость, обработка сбоев, параллелизм, прозрачность, управляемость.**

1) Открытость. 📘 Гарантирует расширяемость, возможность повторного использования, использование открытых протоколов и стандартов. Важны чёткие спецификации и полная документация, опубликованные интерфейсы, тестирование на многих платформах. 💡 Систему можно развивать и подключать к ней чужие компоненты, потому что правила взаимодействия известны всем (как USB-разъём: любое устройство подходит к любому компьютеру).

2) Безопасность. 📘 Физическая распределённость означает возможность доступа злоумышленников к компонентам, поэтому важную информацию нужно посылать по сети безопасно и эффективно. Средства: *аутентификация* (проверка подлинности), *авторизация* (предоставление прав на действия и проверка этих прав), *шифрование данных*. 💡 Аутентификация отвечает на вопрос «кто ты?» (логин и пароль), авторизация — «что тебе разрешено?» (обычный пользователь или администратор), шифрование прячет данные от тех, кто подслушивает сеть.

3) Масштабируемость. 📘 Система масштабируема, если остаётся эффективной при росте числа пользователей или ресурсов. Проблемы: рост стоимости физических ресурсов и потеря производительности. Способы обхода «узких мест»: *децентрализация алгоритмов* (пример — служба имён DNS), *тиражирование* и *кэширование* данных. 💡 Сайт должен одинаково быстро работать и для 100, и для 100 000 посетителей. Для этого нагрузку не сводят в одну точку, а часто запрашиваемые данные хранят «поближе» к пользователю.

4) Обработка сбоев. 📘 Сбои возникают чаще, но обычно носят локальный характер. Обработка включает: *диагностику* (например, ошибки передачи находят по контрольным суммам; но иногда невозможно понять, сервер не работает или просто перегружен), *маскирование* (сбой скрывают от пользователя, но это возможно не всегда) и *восстановление*. 💡 Обнаружить поломку, по

возможности спрятать её от пользователя (переключиться на резервный сервер) и вернуть систему в рабочее состояние.

5) Методы решения проблем параллелизма. Нужны механизмы синхронизации и защиты общих данных (блокировки, транзакции), о которых шла речь в п. 1.4.

6) Прозрачность. 📘 Это сокрытие гетерогенной (разнородной) и распределённой структуры системы так, чтобы пользователю система представлялась монолитной. Степень прозрачности связана с производительностью. 💡 Пользователь не должен знать, на скольких серверах работает сайт и где они стоят.

Вид прозрачности	📘 Что скрывается	💡 Пример
Доступа	Различия в представлении данных и доступе к ресурсам	Файл открывается одинаково, лежит он на Windows или Linux
Расположения	Где находится ресурс	Адрес сайта не говорит, в каком городе сервер
Переноса	Факт перемещения ресурса	Сайт переехал на другой сервер, адрес остался прежним
Смены местоположения	Перемещение ресурса *во время* обработки	Сервис «переехал» посреди сессии, а пользователь не заметил
Репликации	Факт существования нескольких копий	Несколько копий БД выглядят как одна
Параллелизма	Одновременная работа многих процессов	Тысячи покупателей оформляют заказы и не мешают друг другу
Обработки ошибок	Сбои других компонентов, восстановление	Один компонент упал, а пользователь продолжает работу

Критически важными пособие называет прозрачность **доступа**, **расположения** и **параллелизма**.

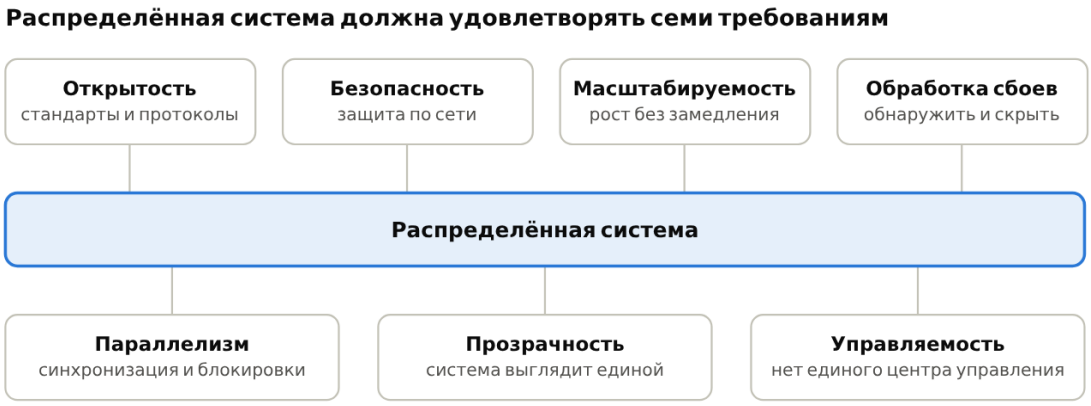


схема · 7 требований

Схема 3. Семь требований к распределённой системе.

7) Управляемость. 📘 У распределённых ресурсов нет центральной точки управления. Локальная оптимизация не всегда означает глобальную; нужно учитывать гетерогенность и другие особенности, а глобальный взгляд на проблему не всегда возможен (бывают системы, никому конкретно не принадлежащие). 💡 Когда компьютеры принадлежат разным людям и стоят в разных странах, настроить всё «как лучше» для всей системы очень трудно: каждый улучшает только свою часть.

1.6. Краткий итог для ответа

1. Распределённая система — автономные узлы, связанные сетью и обменивающиеся сообщениями.
2. Создаются ради совместного использования ресурсов и улучшения характеристик (скорость, надёжность, цена).
3. Принципы построения: функциональное разделение, естественное разделение, балансировка нагрузки.
4. Последствия: параллельность, нет глобального времени, нет глобального состояния, сбои.
5. Требования: открытость, безопасность, масштабируемость, обработка сбоев, параллелизм, прозрачность, управляемость.

Вопрос 2. Распределённая система: сложности реализации, модели, архитектура и принципы, преодоление сложности

2.1. Сложности при реализации

📘 **По пособию.** Выделяются четыре группы сложностей: выбор архитектуры, гетерогенность (разнородность) среды, сложность развёртывания и сложность отладки.

1) Выбор архитектуры. 📘 Важнейшие характеристики системы очень сильно зависят от выбранной архитектуры. Практически невозможно отказаться от решений, принятых в начале проектирования, если они оказались неверными. Решение — использовать широко известные шаблоны (образцы) архитектур.

💡 Архитектура — это фундамент дома. Ошибку в фундаменте нельзя исправить, не разобрав дом. Поэтому архитектуру не изобретают с нуля, а берут проверенные шаблоны.

2) Гетерогенность среды. 📘 В распределённой системе различаются сетевые инфраструктуры, оборудование (например, Intel и Motorola), программное обеспечение (например, UNIX sockets и Winsock calls), языки программирования и представления данных. Компоненты работают на разных платформах, и эти различия должны быть скрыты. Интерфейсы и реализация могут быть разными, но базовые концепции обычно неизменны. Главное средство борьбы с гетерогенностью — стандарты.

💡 Это как розетки в разных странах: форма вилки разная, а суть одна — подать ток. Нужен либо общий стандарт, либо переходник. В программировании роль переходника играет *middleware*.

Что различается	📘 Пример из пособия	💡 Простыми словами
Сетевые инфраструктуры	Разные сети	Узлы подключены разными способами и по разным правилам
Оборудование	Intel и Motorola	Разные процессоры по-разному хранят и обрабатывают данные
Программное обеспечение	UNIX sockets и Winsock calls	Обмен по сети один и тот же, а вызовы в разных ОС разные
Языки программирования	Компоненты на разных языках	Один модуль на Java, другой на C++ — они должны понимать друг друга
Представления данных	Разный формат чисел и строк	Одно и то же значение закодировано по-разному

Два решения проблемы гетерогенности:

- использование распространённых открытых стандартов и протоколов (HTTP, SQL, XML и т. п.);
- использование промежуточного программного обеспечения (**middleware**).

Middleware. 📘 Промежуточный программный слой, который:

- позволяет гетерогенным узлам взаимодействовать;
- определяет однородную вычислительную модель;
- поддерживает один или несколько языков программирования;

- обеспечивает поддержку распределённых приложений: вызов удалённых объектов, удалённый вызов SQL, распределённую обработку транзакций.

Примеры из пособия: **CORBA, JavaEE, Microsoft DCOM**.

💡 Middleware — это переводчик на переговорах. Каждая сторона говорит на своём языке, а переводчик следит, чтобы смысл дошёл без искажений. В Java EE эту роль играет сервер приложений: он берёт на себя сетевой обмен, транзакции и безопасность, а программист пишет только бизнес-логику.



схема · middleware между узлами и приложением

Схема 4. Middleware скрывает различия узлов и даёт приложению единую модель.

3) Сложность развёртывания. 📘 Включает три задачи:

- *фрагментация* — разделение приложения на модули развёртывания;
- *конфигурация* — связь модулей друг с другом (зависимости);
- *размещение* — выгрузка модулей в целевую систему и распределение вычислительных модулей между узлами (статическое или динамическое), с использованием специализированных инструментальных средств.

💡 Вы разобрали конструктор на детали (фрагментация), составили схему, какая деталь к какой подходит (конфигурация), и разложили детали по комнатам (размещение). Статическое размещение — раскладываем один раз заранее; динамическое — переносим детали по ходу работы. В Java EE пример такого модуля — файлы `.war` и `.jar`.

4) Сложность отладки. 📘 Нет глобального состояния. Параллельность приводит к неповторяемости (невоспроизводимости) результатов выполнения. Компоненты распределены по разным узлам.

💡 Ошибка «плавающая»: сегодня она есть, завтра нет — всё зависит от того, чьё сообщение пришло раньше. К тому же нельзя поставить точку останова и «заморозить» сразу все компьютеры, чтобы спокойно разобраться.

2.2. Модели распределённых систем

📘 **По пособию.** Различают модели архитектуры и другие модели.

1) Модель архитектуры. Должна решать две проблемы: *физическое размещение компонентов между узлами* и *взаимодействие между компонентами*. Реальные функции отдельных компонентов в ней не указываются. Указываются:

- расположение (размещение по узлам);
- шаблоны распределения данных и задач по их обработке;
- взаимодействие компонентов (роли компонентов, шаблоны взаимодействия).

💡 Модель архитектуры — план города: видно, где стоят дома и как проложены дороги между ними. Что именно происходит внутри каждого дома, на плане не показано.

2) Другие модели. 📘 Формальное описание различных параметров и свойств системы: модели взаимодействия, обработки ошибок, безопасности и т. д.

💡 Это «правила игры» для системы: как именно узлы передают сообщения (модель взаимодействия), какие сбои считаем возможными (модель ошибок), каких злоумышленников опасаемся и от чего защищаемся (модель безопасности).

Модель	Какой вопрос решает
Архитектурная	Где стоят компоненты и как они связаны?
Взаимодействия	Как передаются сообщения, есть ли общее время?
Ошибок	Какие сбои возможны и как система себя при них ведёт?
Безопасности	От кого и от каких угроз защищаемся?

2.3. Архитектура: что это такое

📘 **По пособию.** Архитектура:

- определяет разделение системы на наиболее крупные составные части;
- определяет конструктивные решения, которые после их принятия с трудом поддаются изменению;
- отображает общий взгляд разработчиков на результаты проектирования системы: идентификацию главных компонентов, способов их взаимодействия, выбор основополагающих решений, не подлежащих изменению в будущем.

💡 Архитектура — это самые крупные и самые «дорогие в переделке» решения: из каких больших блоков состоит система и как они общаются. Мелочи (названия переменных, оформление кнопок) к архитектуре не относятся.

2.4. Основные принципы архитектуры

Пособие называет восемь принципов.

Принцип	📘 Формулировка в пособии	💡 Простыми словами
Согласованность	Частичное знание системы позволяет предсказать остальное	Разобрались, как работает один раздел, — угадаете, как работают остальные
Ортогональность	Функции независимы и специфицированы по отдельности	Как руль и педаль газа: каждый делает своё и не мешает другому
Соответствие	Только функции, отвечающие существенным требованиям; нет ненужных	Нет «лишних кнопок», которыми никто не пользуется
Экономичность	Отсутствие дублирования	Одно и то же не делается дважды в разных местах
Прозрачность	Функции должны быть известны пользователю	Пользователь может узнать, что система умеет
Общность	Функция вводится так, чтобы отвечать как можно большему числу назначений	Лучше один универсальный инструмент, чем десять узких
Открытость	Можно использовать функцию иначе, чем предполагалось при проектировании	Функцию можно применить творчески, в непредусмотренной ситуации
Полнота	Функции с учётом ограничений как можно полнее соответствуют требованиям и пожеланиям пользователя	Система закрывает потребности пользователя, насколько позволяют деньги и технологии

2.5. Преодоление сложности

Пособие предлагает два основных способа.

Способ 1. Типовые решения — образцы проектирования (design patterns).

📘 Каждое типовое решение описывает некую повторяющуюся проблему и ключ к её разгадке, причём так, что этим ключом можно пользоваться многократно, ни разу не придя к одному и тому же результату. Структура типового решения:

1. название решения;

2. назначение (аннотация);
3. мотивация, применимость;
4. принцип действия (структура, участники, отношения);
5. результаты;
6. реализация.

💡 Образец проектирования — как кулинарный рецепт: у блюда есть название, понятно, когда его готовить, какие нужны ингредиенты и что получится. Рецептом пользуются снова и снова, а блюдо каждый раз выходит со своим вкусом.

Способ 2. Разделение системы на слои (расслоение).

📘 Основная идея: независимость нижележащих уровней от вышележащих. Основная задача: уменьшать сложность систем, разделяя их на слои и сервисы.

- **Слой (уровень)** — группа сильно связанных и закрытых элементов, реализующих одну функциональность.
- **Сервис** — функциональность, обеспечиваемая для вышестоящего слоя.

💡 Это как этажи здания или звенья производства. Каждый слой знает только о соседнем: покупателю в магазине не нужно знать, как устроен склад, ему важно, что товар выдают на кассе.

Пособие показывает три варианта на примере прикладной системы с тремя видами логики: *логика представления* (интерфейс), *бизнес-логика* (правила предметной области) и *логика доступа к данным*.

Вариант	Рисунок в пособии	Как устроен
Монолитная система	Рисунок 22	Вся логика — единый блок, который напрямую работает с базой данных
Двухслойная система	Рисунок 23	Логика представления и бизнес-логика вместе; отдельно — логика доступа к данным и БД
Трёхслойная система	Рисунок 24	Каждая логика выделена в свой слой: представление, бизнес-логика, доступ к данным

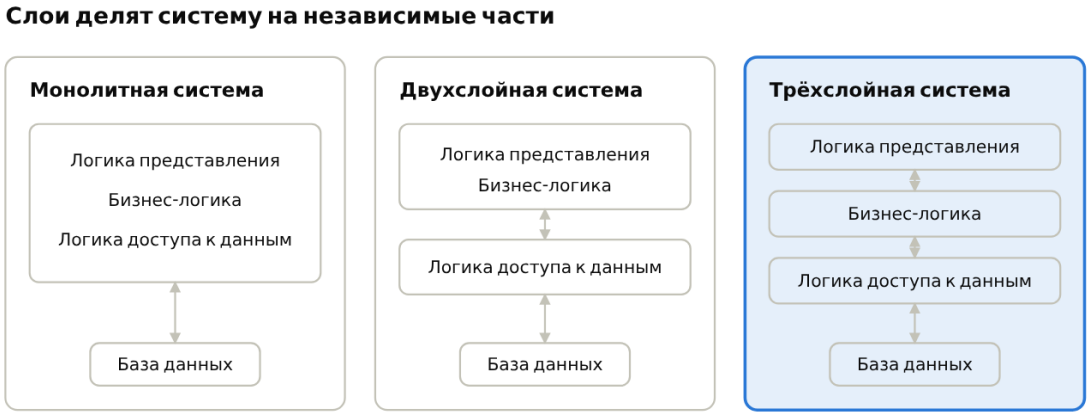


схема · монолит, два слоя, три слоя (рис. 22–24 пособия)

Схема 5. Монолитная, двухслойная и трёхслойная системы (по рисункам 22–24 пособия). Стрелки — обмен данными между слоями.

Три основных слоя (📘 по пособию):

- **представление** (presentation);
- **домен** (domain) — предметная область, бизнес-логика;
- **работа с данными** (data source).

Слой и ярус. 📘 Рекомендуется различать *слой* (layer) — логическое разделение, и *уровень, или ярус* (tier) — физическое разделение. 💡 Слой — это разделение кода по ролям (в проекте это могут быть разные пакеты), ярус — разделение по компьютерам (разные серверы). Три слоя можно запустить и на одной машине, и на трёх.

2.6. Какую архитектурную модель выбрать

📘 Пособие подчёркивает: выбор модели архитектуры зависит от особенностей задачи. Названы четыре модели; ниже кратко расшифрованы (пояснения — дополнение к конспекту).

Модель	Суть	💡 Пример
Клиент — сервер	Клиент отправляет запрос, сервер его обрабатывает и возвращает ответ	Браузер и веб-сервер
Предоставление услуг пулом серверов	Одну услугу оказывает не один, а несколько серверов	Несколько серверов интернет-магазина за одним адресом
Прокси- и кэш-серверы	Между клиентом и сервером стоит посредник, который хранит копии часто запрашиваемых данных	Кэш в браузере или в сети провайдера, который отдаёт уже скачанную картинку

Модель	Суть	💡 Пример
Модель равных процессов (peer-to-peer)	Все узлы равноправны: каждый и клиент, и сервер	Файлообменные сети, мессенджеры с прямым соединением

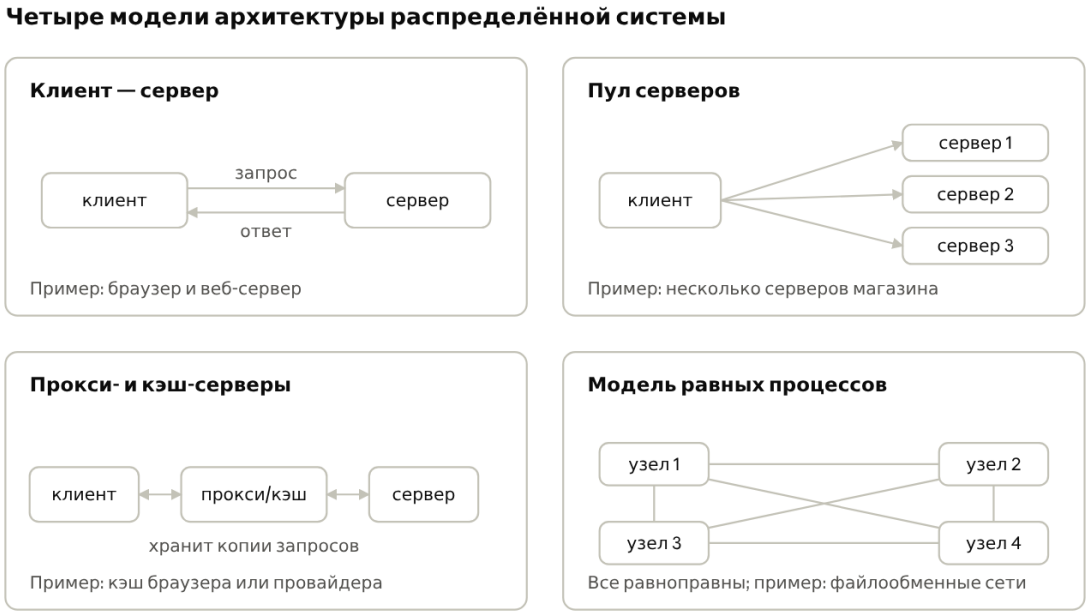


схема · 4 модели архитектуры

Схема 6. Четыре модели архитектуры: кто с кем и как взаимодействует.

2.7. Краткий итог для ответа

1. Сложности реализации: выбор архитектуры, гетерогенность, развёртывание, отладка.
2. Гетерогенность преодолевается стандартами и middleware (CORBA, Java EE, DCOM).
3. Модель архитектуры отвечает на два вопроса: где размещены компоненты и как они взаимодействуют. Есть и другие модели: взаимодействия, ошибок, безопасности.
4. Архитектура — крупнейшие и трудно изменяемые решения; её принципы: согласованность, ортогональность, соответствие, экономичность, прозрачность, общность, открытость, полнота.
5. Сложность снижают образцы проектирования и разделение на слои (представление, бизнес-логика, доступ к данным); слой — логическое разделение, ярус — физическое.

Вопрос 3. JDBC: типы драйверов, цели интерфейса, примеры использования, основы программирования, подготовленные команды, метаданные

3.1. Что такое JDBC

📘 **По пособию.** В 1996 году компания Sun выпустила первую версию интерфейса для организации доступа Java-приложений к базам данных — **JDBC (Java DataBase Connectivity)**. Он позволяет соединяться с БД, запрашивать и обновлять данные с помощью языка **SQL**, который фактически стал стандартным средством взаимодействия с реляционными базами данных. Программы, созданные с помощью Java и JDBC, не зависят от используемой платформы и поставщика программного обеспечения. В JDK 1.2 (1998 год) была включена версия 2 JDBC, в JDK 1.4 и 1.5 — версия 3 и т. д.

💡 **Простыми словами.** JDBC — это «универсальная розетка» между Java-программой и базой данных. Программист пишет код один раз, а производитель БД (PostgreSQL, Oracle, MySQL и др.) даёт «вилку» — драйвер. Если заменить базу данных, код программы почти не меняется — меняется только драйвер и адрес подключения.

3.2. Как устроен JDBC: два интерфейса и диспетчер драйверов

📘 **По пособию.** Поставщики БД поддерживали идею стандартного сетевого протокола доступа к БД, но каждый хотел взять за основу свой протокол. В итоге согласились, что специалисты Sun создали Java API для SQL-доступа к данным и *диспетчер драйверов*, позволяющий подключать драйверы независимых производителей. В результате создано **два интерфейса**:

- **JDBC API** — для разработчиков приложений;
- **JDBC Driver API** — для поставщиков БД и инструментальных средств.

Подход основан на модели **ODBC (Open Database Connectivity)** — интерфейса доступа к БД, разработанного фирмой Microsoft. Программы, соответствующие API, взаимодействуют с диспетчером драйверов JDBC (`DriverManager`), который использует присоединённые драйверы для общения с БД.

💡 **Простыми словами.** У JDBC две «стороны». Программист общается только с первой (JDBC API): «открой соединение», «выполни запрос». Производитель БД реализует вторую (JDBC Driver API): как именно «разговаривать» с его базой. `DriverManager` — диспетчер, который по адресу подключения находит нужный драйвер.

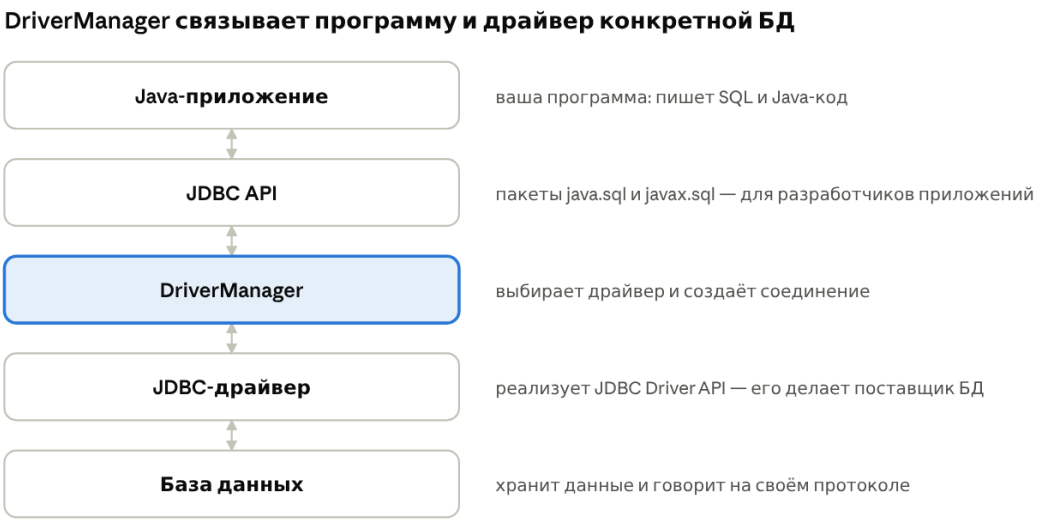


схема · архитектура JDBC

Схема 7. Слои JDBC: разработчик видит только JDBC API, а конкретная БД скрыта за драйвером.

3.3. Типы JDBC-драйверов

📘 Пособие выделяет четыре типа драйверов.

Тип	📘 Суть по пособию	💡 Простыми словами
Тип 1	Транслирует JDBC в ODBC и использует драйвер ODBC. В JDK входил один такой драйвер — мост JDBC/ODBC. Нужно установить и настроить ODBC-драйвер. В первом выпуске JDBC мост предполагался только для тестирования, а не для рабочего применения	«Переводчик через переводчика»: Java говорит с мостом, мост — с ODBC, ODBC — с базой. Много звеньев, нужна настройка на каждом компьютере. В Java 8 мост JDBC-ODBC удалён
Тип 2	Создаётся преимущественно на Java и частично на собственном языке программирования для общения с клиентским API БД. Нужно установить код, специфичный для платформы	Наполовину Java, наполовину «родной» код под конкретную ОС: привязан к платформе и требует установки
Тип 3	Создаётся только на основе	Чистая Java. Клиент говорит на

Тип	📘 Суть по пособию	💡 Простыми словами
	библиотеки Java и использует независимый от БД протокол взаимодействия клиента и сервера. Код, зависящий от БД, находится только на сервере, поэтому доставка программ упрощается	общем «нейтральном» языке с сервером-посредником, а тот уже общается с конкретной БД. Клиентам ничего ставить не надо
Тип 4	Java-библиотека, которая транслирует JDBC-запросы непосредственно в протокол конкретной БД	Чистая Java, никаких посредников: драйвер сам «говорит» на языке базы. Самый простой для подключения вариант (таким является драйвер PostgreSQL, использованный в примерах пособия)

📘 Большинство поставщиков БД применяют драйверы типа 3 или 4. Кроме того, некоторые компании специализируются на создании драйверов, обеспечивающих более полное соответствие стандартам, поддержку большего числа платформ, более высокую производительность или надёжность, чем драйверы производителей СУБД.

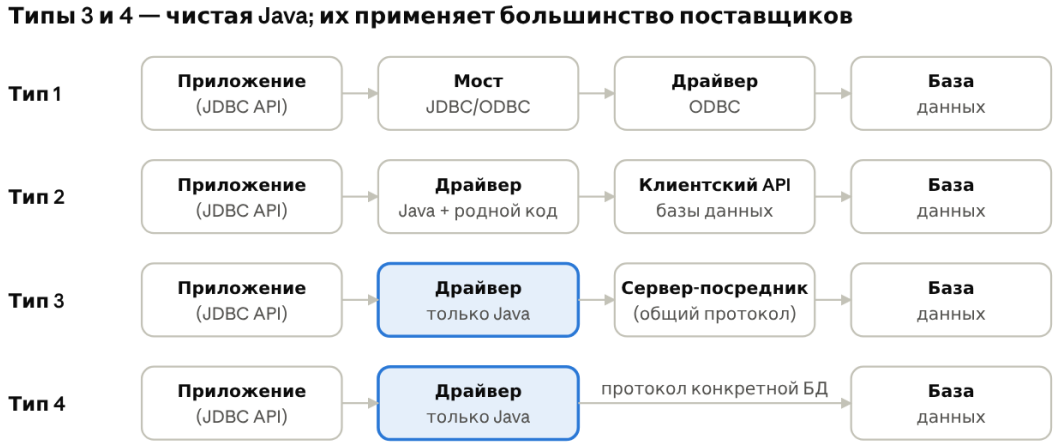


схема · 4 типа драйверов

Схема 8. Четыре типа JDBC-драйверов: какие звенья стоят между приложением и базой данных.

3.4. Основные цели интерфейса JDBC

📘 **По пособию.** Цели можно сформулировать так:

1. Разработчики создают программы на языке Java, пользуясь для доступа к БД стандартными средствами языка SQL, при этом следуют только соглашениям языка Java.
2. Производители СУБД и инструментальных средств поставляют драйверы только низкого уровня.

💡 **Простыми словами.** Программисту не нужно знать особенности каждой базы данных: он пишет обычный SQL и обычный Java-код. А «тяжёлая» работа по общению с конкретной БД лежит на драйвере, который делает производитель базы.

3.5. Примеры использования JDBC

Модель 1. Традиционная клиент/сервер (рис. 25 пособия). Графический интерфейс реализован на стороне клиента, а БД расположена на стороне сервера. Драйвер JDBC находится на стороне клиента и общается с сервером БД по протоколу БД.

Модель 2. Трёхуровневая (рис. 26 пособия). 📘 Современная тенденция — переход от клиент/сервера к «трёхуровневой модели»: клиент не формирует обращений к БД, а обращается к средствам *промежуточного уровня* на сервере, который уже выполняет запросы к БД. Модель отделяет визуальное представление (на компьютере-клиенте) от бизнес-логики (промежуточный уровень) и данных (в БД). Поэтому к тем же данным и тем же бизнес-правилам можно обращаться клиентами разных типов: Java-приложениями, апплетами, веб-формами. Клиент и промежуточный уровень общаются по HTTP (если клиент — браузер), через RMI (приложения или апплеты) или другим способом. **JDBC используется для связи промежуточного уровня с БД.**

💡 Это как в банке: клиент (в зале или в мобильном приложении) никогда не заходит в хранилище сам. Он обращается к операционисту (промежуточный уровень), а тот по правилам банка идёт в хранилище (БД). Благодаря этому разные клиенты — окно, банкомат, приложение — работают по одним и тем же правилам. В Java EE на промежуточном уровне работают сервлеты, JSP и Enterprise-компоненты, и именно они обращаются к БД через JDBC.

↗ схема · рис. 25 и 26 пособия

Схема 9. Двухуровневая и трёхуровневая модели (по рисункам 25 и 26 пособия).

3.6. Основы программирования JDBC

📘 Классы для программирования JDBC находятся в пакетах `java.sql` и `javax.sql`.

Шаг 1. Регистрация драйвера. 📘 `DriverManager` отвечает за выбор драйверов и создание соединения, но перед этим драйвер надо зарегистрировать. Три способа:

```
// 1) из командной строки
java -Djdbc.drivers=org.postgresql.Driver MyClass

// 2) системным свойством в программе
System.setProperty("jdbc.drivers", "org.postgresql.Driver");

// 3) вручную, загрузив класс драйвера
Class.forName("org.postgresql.Driver");
```

📘 Третий способ применяют, если диспетчер не может загрузить драйвер из-за ограничений конкретного драйвера или если программа выполняется в среде контейнера, например, сервлета.

Шаг 2. Соединение с БД. 📘 База данных задаётся URL. Для PostgreSQL возможны три формы:

- `jdbc:postgresql:database`
- `jdbc:postgresql://host/database`
- `jdbc:postgresql://host:port/database`

```
Connection connection = DriverManager.getConnection(
    "jdbc:postgresql://localhost:5432/MyTest", "postgres", "root");
```

💡 URL — это «адрес» базы: протокол (`jdbc:postgresql`), компьютер (`localhost`), порт (`5432`) и имя базы (`MyTest`). Далее передаются логин и пароль.

Файл свойств (пример 49). 📘 Драйвер, URL, имя и пароль удобно хранить в файле свойств:

```
// database.properties:
// jdbc.drivers = org.postgresql.Driver
// jdbc.url = jdbc:postgresql://localhost:5432/MyTest
// jdbc.username = postgres
// jdbc.password = root

Properties props = new Properties();
FileInputStream in = new FileInputStream("database.properties");
props.load(in);
in.close();
String drivers = props.getProperty("jdbc.drivers");
if (drivers != null) System.setProperty("jdbc.drivers", drivers);
String url = props.getProperty("jdbc.url");
String username = props.getProperty("jdbc.username");
String password = props.getProperty("jdbc.password");
Connection connection = DriverManager.getConnection(url, username, password);
```

💡 Зачем: чтобы при смене пароля или адреса сервера не перекомпилировать программу — достаточно поправить текстовый файл.

Шаг 3. Выполнение запроса (пример 50). 📘 Метод `getConnection()` возвращает объект `Connection`; с его помощью создаётся `Statement`, выполняющий SQL-команды. Метод `executeQuery()` возвращает `ResultSet` — результат, который просматривается построчно.

```
Statement statement = connection.createStatement();
ResultSet resultSet = statement.executeQuery("SELECT * FROM item");
while (resultSet.next()) {
    String str = resultSet.getString("name");
    System.out.println(str);
}
```

Важные детали работы с `ResultSet` (📘 по пособию):

- Курсор в начале работы стоит **перед** первой строкой, поэтому надо вызвать `next()`, чтобы перейти к первой.
- Порядок строк произвольный; если он важен, указывается `ORDER BY`.

- Для каждого типа Java есть свой метод получения значения (`getString()`, `getDouble()` и т. д.), у каждого два варианта: по **номеру** столбца (нумерация с **1**) или по **имени** столбца: `resultSet.getString(1)`, `resultSet.getDouble("Price")`.
- Если запрошенный тип не совпадает с фактическим, выполняется преобразование (например, число из столбца `Price` можно получить как строку).

💡 `ResultSet` — это «курсор», бегущий по таблице результата. Сначала он стоит «над» таблицей; `next()` опускает его на следующую строку и возвращает `false`, когда строки закончились.

Соответствие типов SQL и Java (таблица 3 пособия):

Тип SQL	Тип Java
INTEGER, INT	`int`
SMALLINT	`short`
NUMERIC(m,n), DECIMAL(m,n), DEC(m,n)	`java.math.BigDecimal`
REAL	`float`
DOUBLE	`double`
CHARACTER(n), CHAR(n), VARCHAR	`String`
BOOLEAN	`boolean`
DATE	`java.sql.Date`
TIME	`java.sql.Time`

Изменение данных. 📘 Метод `executeUpdate()` служит для команд `INSERT`, `UPDATE`, `DELETE`, а также команд определения данных (`CREATE TABLE`, `DROP TABLE`). Он возвращает число строк, затронутых командой.

```
int n = statement.executeUpdate("UPDATE item SET price = 100 WHERE id = 1");
```

Шаг 4. Закрытие ресурсов. 📘 После работы с `ResultSet`, `Statement` и `Connection` нужно вызвать `close()`: эти объекты используют большие структуры данных, и не стоит ждать сборщика мусора. `close()` у `Statement` закрывает связанные с ним `ResultSet`; `close()` у `Connection` закрывает все `Statement` этого соединения. Если соединение живёт недолго, `close()` помещают в блок `finally`, чтобы соединение гарантированно не осталось открытым.

💡 Это как выключать свет и закрывать кран: пока соединение открыто, оно занимает ресурсы и сервера, и вашей программы. (В современных версиях Java для этого удобен `try-with-resources`, но в пособии описан вариант с `finally`.)

Порядок работы — коротко: зарегистрировать драйвер → получить `Connection` → создать `Statement` → выполнить запрос (`executeQuery`/`executeUpdate`) → обработать `ResultSet` → закрыть всё в `finally`.

↗ схема · порядок работы с JDBC

Схема 10. Основная цепочка объектов JDBC и дополнительные возможности: подготовленные запросы и метаданные.

3.7. Предварительно подготовленные команды (PreparedStatement)

📘 **По пособию.** Вместо отдельной SQL-команды для каждого запроса следует подготовить запрос с подстановочной переменной и многократно использовать его, меняя только значение переменной. Это существенно повышает эффективность. Перед каждым выполнением запроса СУБД строит *план его эффективного исполнения*; подготовленный запрос позволяет не создавать план заново. Подстановочная переменная обозначается символом `?`.

```
String s = "SELECT name, price FROM tovar WHERE price > ? AND price < ?";
PreparedStatement ps = connection.prepareStatement(s);
ps.setDouble(1, 2000);
ps.setDouble(2, 6000);
ResultSet rs1 = ps.executeQuery();
```

- Перед выполнением значения `?` связываются с фактическими с помощью методов `set…()`; для разных типов — разные методы (`setDouble()`, `setString()`, `setInt()` и т. д.).
- Первый параметр — **номер** подстановочной переменной (с 1), второй — её значение.
- При повторном использовании связи сохраняются; `set…()` вызывают только для тех переменных, которые изменились.
- Для изменения данных используется `executeUpdate()`.

💡 **Простыми словами.** `PreparedStatement` — это «шаблон письма» с пропусками (`?`). Вы один раз показываете базе шаблон, она «разбирается», как его выполнять быстрее всего, а потом вы лишь подставляете значения в пропуски: «от 2000 до 6000», затем «от 100 до 500» и т. д. База не тратит время на повторный разбор.

	`Statement`	`PreparedStatement`
Запрос	Текст каждый раз целиком	Шаблон с `?`, значения подставляются
План выполнения	Строится при каждом выполнении	Готовится заранее, переиспользуется
Удобство	Подходит для разовых простых команд	Подходит для повторяющихся запросов с разными значениями

К пособию можно добавить (это дополнение): подстановка параметров через `set…()` также защищает от SQL-инъекций, потому что значение передаётся отдельно от текста команды и не может изменить её смысл.

3.8. Метаданные

📘 **По пособию.** JDBC позволяет получать информацию о структуре таблиц и самой БД: список всех таблиц, имена и типы столбцов. Это нужно, если пишется программа для работы с произвольной БД. В SQL информация о структуре БД и её компонентов называется **метаданными**.

💡 **Простыми словами.** Данные — это содержимое таблиц (товары, цены). Метаданные — «данные о данных»: как называются таблицы, какие в них столбцы, какого они типа. Это как оглавление и описание полей в книге.

1) `DatabaseMetaData` — сведения о БД.

```
DatabaseMetaData meta = connection.getMetaData();
ResultSet mrs = meta.getTables(null, null, null, new String[] {"TABLE"});
```

📘 Метод `getTables(String catalog, String schemaPattern, String tableNamePattern, String types[])` возвращает описание всех таблиц каталога, соответствующих шаблонам схемы и имён, и заданным типам.

- **Схема** — группа связанных таблиц и полномочий доступа; **каталог** — связанная группа схем.
- `catalog` и `schemaPattern` могут быть пустыми строками (для таблиц без каталога и схемы) или `null` (не учитывать каталог/схему).
- `types` — массив типов таблиц: `TABLE`, `VIEW`, `ALIAS` и т. д.; если `null`, возвращаются таблицы всех типов.

Результат содержит 5 столбцов типа `String` (таблица 4 пособия):

№	Столбец	Содержание
1	TABLE_CAT	Каталог таблицы (может быть `null`)
2	TABLE_SCHEM	Схема (может быть `null`)
3	TABLE_NAME	Имя таблицы
4	TABLE_TYPE	Тип таблицы
5	REMARKS	Комментарии к таблице

Чтобы получить имя таблицы (третий столбец), вызывают `mrs.getString(3)`.

2) `ResultSetMetaData` — сведения о наборе результатов.

```
ResultSet rs = statement.executeQuery("SELECT * FROM item");
ResultSetMetaData rsmd = rs.getMetaData();
int count = rsmd.getColumnCount();
String label = rsmd.getColumnLabel(1);
```

📘 После выполнения запроса можно узнать количество столбцов (`getColumnCount()`), имена столбцов (`getColumnLabel(int column)`) и т. д.

💡 Пример применения: универсальная программа, которая выводит любую таблицу на экран, не зная заранее, сколько в ней столбцов и как они называются.

3.9. Краткий итог для ответа

1. JDBC — стандартный интерфейс доступа Java-программ к реляционным БД через SQL; независим от платформы и производителя БД.

2. Два интерфейса: JDBC API (для разработчиков) и JDBC Driver API (для поставщиков БД); посредник — `DriverManager` (по модели ODBC).
3. Четыре типа драйверов: 1 — мост JDBC/ODBC, 2 — Java + родной код, 3 — чистая Java с общим протоколом и сервером-посредником, 4 — чистая Java, напрямую по протоколу БД; чаще применяют типы 3 и 4.
4. Цели: разработчик пишет на Java и SQL, производители поставляют низкоуровневые драйверы.
5. Использование: двухуровневая (клиент/сервер) и трёхуровневая модели; в трёхуровневой JDBC связывает промежуточный уровень с БД.
6. Основной порядок: драйвер → `Connection` → `Statement` → `ResultSet` → `close()`; `executeQuery` для выборки, `executeUpdate` для изменений.
7. `PreparedStatement` — запрос с параметрами `?`, план строится один раз, повышается эффективность.
8. Метаданные: `DatabaseMetaData` (о БД и таблицах) и `ResultSetMetaData` (о результате запроса).